

23AID201: MODELLING, SIMULATION, & ANALYSIS

Term project report on

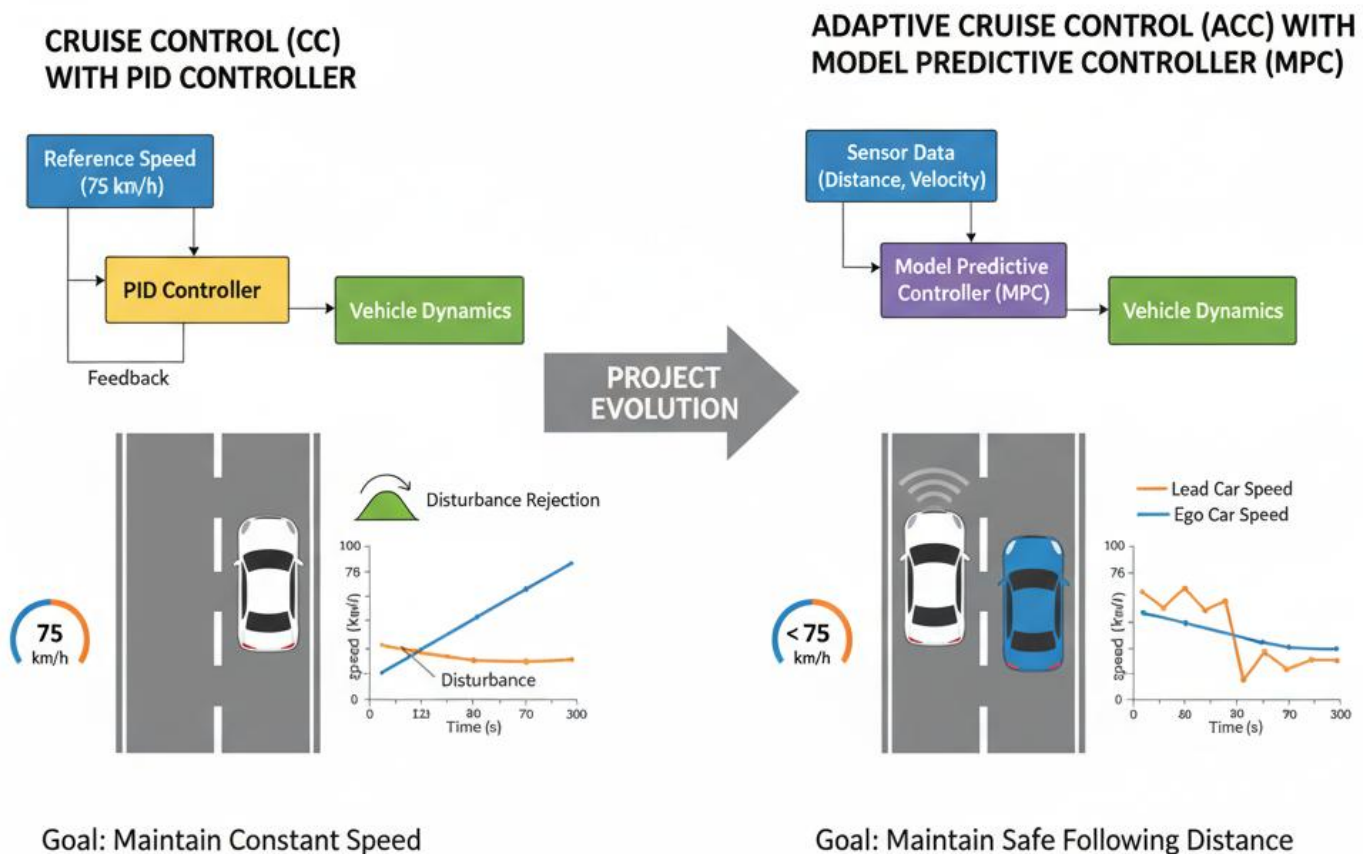
Cruise and adaptive cruise control in cars

by Team 1

Pranav Kumar Reddy	–	CB.AI.U4AID24069
K. Rajesh Kumar Reddy	–	CB.AI.U4AID24067
Dheeraj kumar	–	CB.AI.U4AID24017
Lalith Sagar	–	CB.AI.U4AID24009

Project Abstract:

A driver assistance system called cruise control automatically adjusts the speed of the vehicle without the driver having to continuously press the throttle. MATLAB/Simulink was used to model and simulate a cruise control system in the project's first phase [1]. To maintain the intended vehicle speed under a variety of circumstances, including resistive forces and road slope disturbances, a PID controller was created. The simulation results show fast disturbance rejection, low overshoot, and efficient speed tracking. The development of more sophisticated systems, such as Adaptive Cruise Control (ACC), is made possible by this preliminary work. Building on this, the project was expanded to include designing an ACC system that uses a Model Predictive Controller (MPC) to maintain a safe following distance from a lead vehicle in addition to controlling speed [4].



Introduction:

Modern automobiles demand features that enhance comfort, fuel efficiency, and reduce driver fatigue. Cruise control addresses these needs by automatically maintaining a set speed without manual throttle adjustments [1]. In a scenario like a long stretch of highway, it becomes a tedious task for a driver to manually maintain a constant speed. The cruise control system gives the driver the freedom to lift their foot from the accelerator pedal, and the vehicle will continue at the desired constant speed. For this project's first phase, a PID controller is used in a closed-loop environment to hold the vehicle's speed constant at a setpoint of 75 km/h.

While standard cruise control is effective on open roads, it does not account for changing traffic conditions. This limitation led to the second phase of this project: the development of an Adaptive Cruise Control (ACC) system [2]. This advanced system builds upon the initial model to create a more intelligent driver-assistance feature capable of adjusting speed in response to other vehicles [5].

Problem statement:

The initial problem was to design and simulate a cruise control system that maintains a vehicle's speed at a desired value despite external disturbances such as aerodynamic drag, rolling resistance, and road slope. The project was then extended to address the limitations of standard cruise control by developing an ACC system. The enhanced problem is to design a system that can not only maintain a set speed but also detect a slower-moving lead vehicle and automatically adjust the speed to maintain a safe following distance [2],[5].

This project is highly relevant to the course on modelling, simulation and analysis, as it requires control system design, vehicle dynamics modelling and performance evaluation.

Background about the chosen problem:

Modern automotive engineering increasingly focuses on driver assistance systems that enhance comfort, fuel efficiency, and safety. Among these, **Cruise Control (CC)** stands as a foundational technology, first introduced in the 1950s, designed to automatically maintain a vehicle's set speed through feedback control, typically employing a Proportional-Integral-Derivative (PID) controller [1]. While highly effective on open roads, standard CC does not account for dynamic traffic conditions. This limitation led to the development of **Adaptive Cruise Control (ACC)**, a more advanced system that utilizes sensors to detect lead vehicles and autonomously adjust speed to maintain a safe following distance. ACC forms a stepping

stone toward higher levels of vehicle autonomy, which has been demonstrated in large-scale projects such as the DARPA Urban Challenge [3]. ACC systems often leverage advanced control strategies, such as **Model Predictive Control (MPC)**, to manage multiple objectives and handle safety constraints more effectively, marking a significant step towards fully autonomous driving [3],[4]. This project explores both these paradigms, beginning with a PID-based CC and extending to an MPC-based ACC, to demonstrate the evolution and increasing sophistication of vehicle control systems.

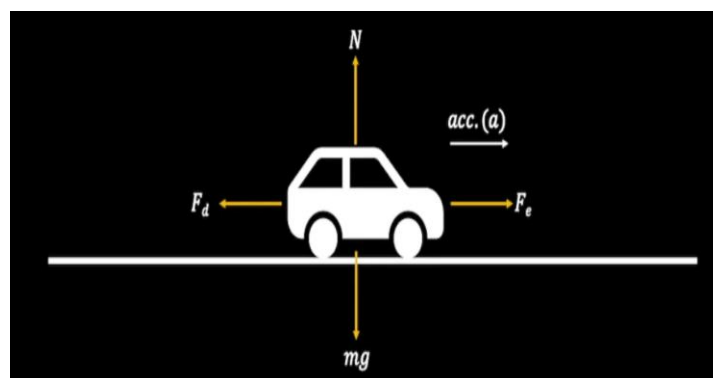
Literature survey

Cruise control is widely studied as a fundamental automotive control problem, often modeled as a first-order system where engine force balances resistive forces like aerodynamic drag [1]. The use of a PID controller is the most common approach due to its simplicity and effectiveness in regulating speed with minimal error. Recent studies have extended this concept to Adaptive Cruise Control (ACC), which uses sensors to adjust speed and maintain safe headway in traffic [2],[5]. Advanced methods like Model Predictive Control (MPC) have been explored to handle safety constraints and improve disturbance rejection. For the scope of a standard cruise control system, a PID-based closed-loop design remains a practical and effective solution [4].

Mathematical model

Cruise control system:

So the car will have a mass **m** because of gravity a force of **mg** acts downwards. Since the road is rigid it will provide an equal and opposite force that is the normal force in the upward direction. The engine will produce a force **F_e** this is called engine force. This is produced due to combustion. **F_d** is the aerodynamic drag which is experienced by the vehicle as the vehicle moves forward and wind hits the vehicle in the opposite direction. The vehicle longitudinal dynamics are commonly represented by Newton's second law, balancing engine force with resistive forces such as drag and rolling resistance [1].



Mg- Force due to gravity, **N**- Normal Reaction, **Fe**- Engine Force, **Fd**- Drag Force.

$$N = mg$$

$$F_e - F_d = ma$$

$$F_e \propto \text{throttle position} \propto \text{input}$$

$$F_e = k_1 \cdot (\text{input to throttle})$$

$$F_d = k_2 v$$

$$k_1 \cdot (\text{input}) - k_2 v = ma$$

$$m \frac{dv}{dt} = -k_2 v + k_1 \cdot (\text{input})$$

The **normal force** and **mg** cancels out each other. Since the vehicle effectively moves forward the resultant force will be acting in the forward direction. Here the **Fe** which is produced by the engine will be proportional to the throttle position of the engine.

Throttle is something that allows air to flow into the engine based on that the amount of fuel injected into the combustion chamber is decided. As a result if we increase the throttle the power or force produced by the engine increases. Here throttle position is directly proportional to input. Input is the error signal which we get from comparing the desired set speed and the speed of the vehicle. After substitution and simplification. We get the given below differential equation.

The vehicle dynamics can be represented as:

$$dv/dt = (k_1/m) \cdot (\text{input}) - (k_2/m) \cdot (v) \quad [1]$$

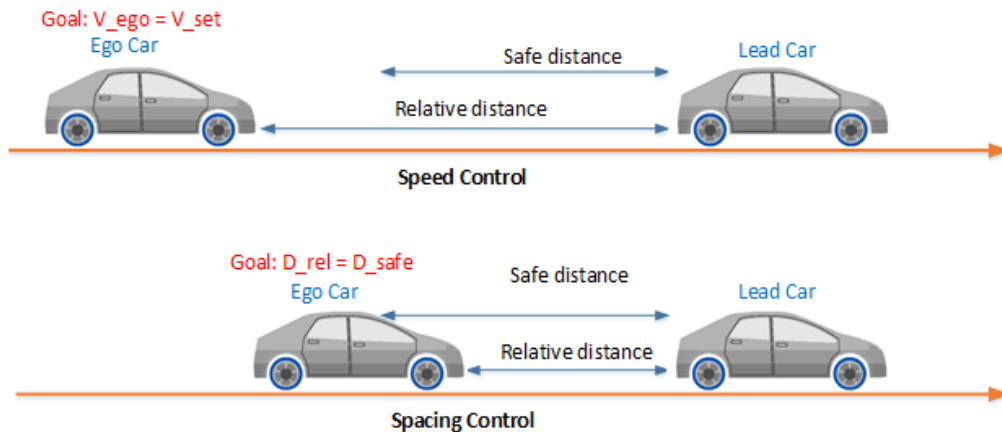
$$m=1000$$

$$k_1=2, k_2=100$$

This first-order system is well-suited for classical control design.

Adaptive cruise control system:

A vehicle (ego car) equipped with adaptive cruise control (ACC) has a sensor, such as radar, that measures the distance to the preceding vehicle in the same lane (lead car), D_{rel} . The sensor also measures the relative velocity of the lead car, V_{rel} . The ACC system operates in the following two modes:



- **Speed control:** The ego car travels at a driver-set speed.
- **Spacing control:** The ego car maintains a safe distance from the lead car.

The ACC system decides which mode to use based on real-time radar measurements. For example, if the lead car is too close, the ACC system switches from speed control to spacing control. Similarly, if the lead car is further away, the ACC system switches from spacing control to speed control. In other words, the ACC system makes the ego car travel at a driver-set speed as long as it maintains a safe distance.

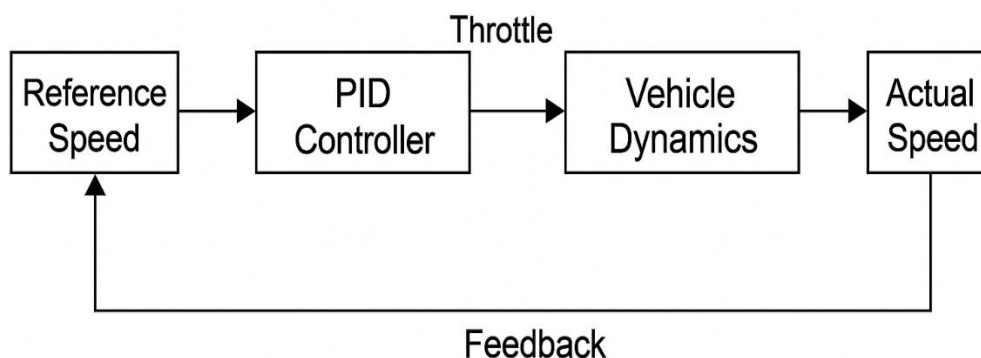
The following rules are used to determine the ACC system operating mode:

- If $D_{rel} \geq D_{safe}$, then speed control mode is active. The control goal is to track the driver-set velocity, V_{set} .
- If $D_{rel} < D_{safe}$, then spacing control mode is active. The control goal is to maintain the safe distance, D_{safe} .

Proposed solution / Chosen method:

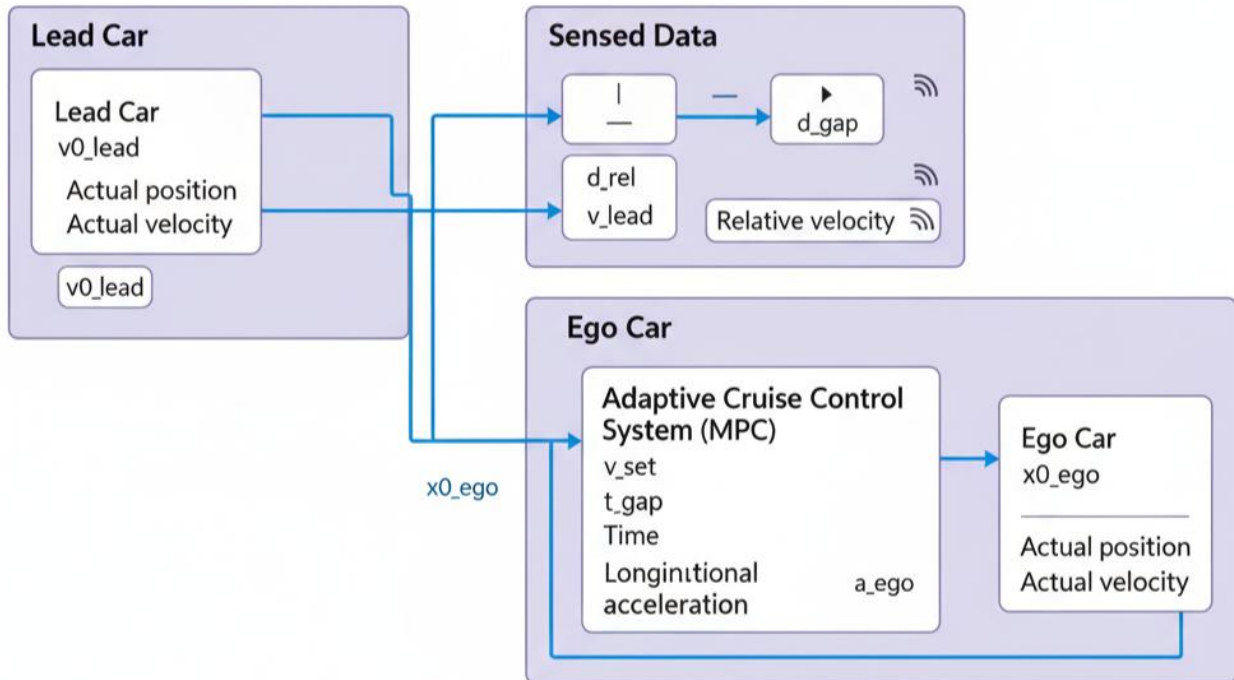
1. **Cruise Control (PID Controller):** A PID Controller was chosen for the standard cruise control system because it ensures a fast response and stable tracking [1]. The integral action eliminates steady-state error, while the derivative action reduces overshoot. The system follows the feedback structure shown below, where the PID controller adjusts the throttle based on the error between the reference and actual speed.

Block Diagram:



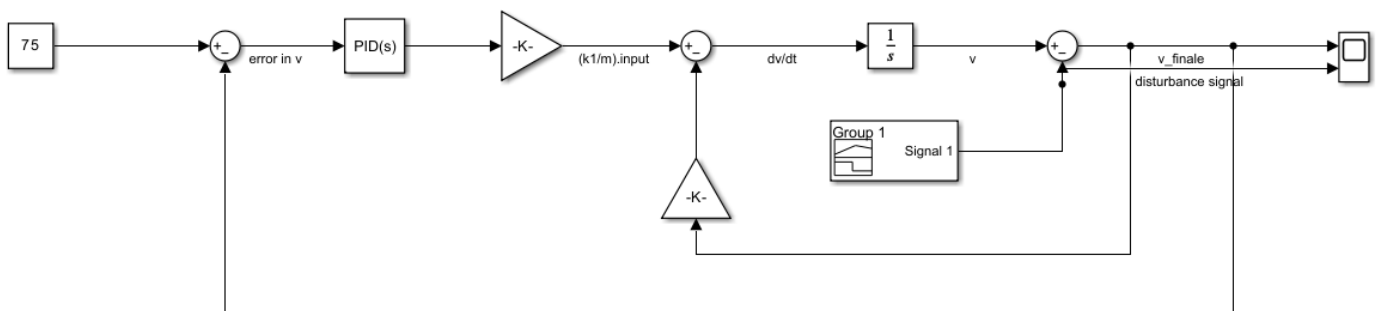
2. **Adaptive Cruise Control (ACC):** Model Predictive Controller (MPC) is used for the more complex ACC system, an MPC was chosen as an advancement, a direction suggested by modern research [4]. MPC is better suited for the ACC problem because it can explicitly handle safety constraints (like minimum following distance) and its predictive nature allows it to plan optimal control actions for smoothly managing the distance to a lead car [4].

Block Diagram:



Implementation details:

Cruise control:



When a vehicle encounters a slope its speed would get disturbed [1]. It would increase or decrease depending upon which type of slope it is [1]. If it is an upward slope the velocity or speed tends to decrease. If it is a downward slope the velocity or speed tends to increase. So here we simulated the slope using change in velocity.

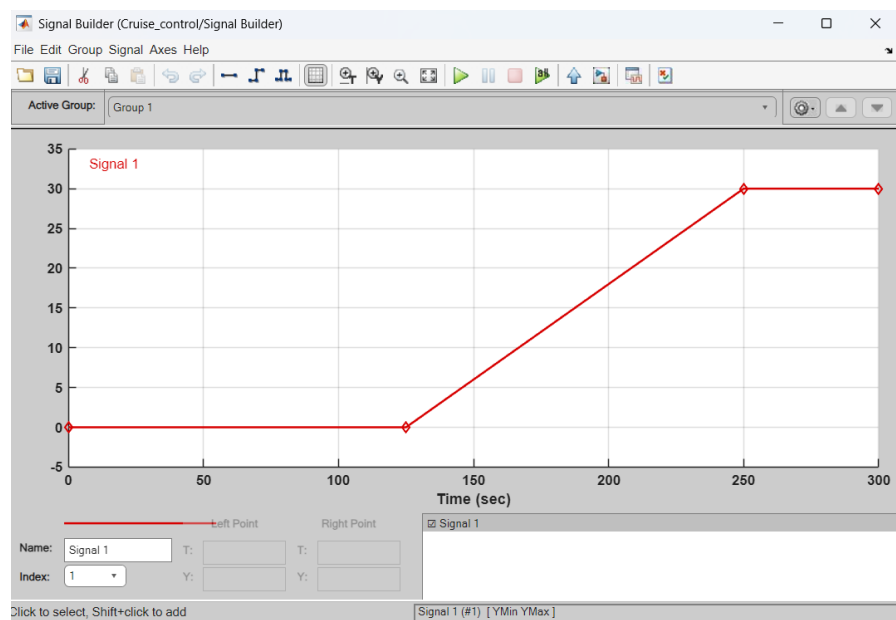
To the integrator sending dv/dt produces v . The signal is coming with a gain of k_2/m so we need a gain block. The value of the gain block will be k_2/m . All the values of m , k_1 , k_2 are obtained by hidden and trail. These are the values for which the system perfectly.

We need a scope for observing speed or velocity of the vehicle. The sign of the sum block is changed to $+-$ because one component it is adding and other it is subtracting.

The input is provided by the PID controller which is the output of the PID controller. So we need a PID block.

We need another gain block as the input is being multiplied by k_1/m . We need an another sum block to take the difference between final and reference velocity. This error will be given as a input to the PID. A constant block is needed for a set speed that is 75kmph.

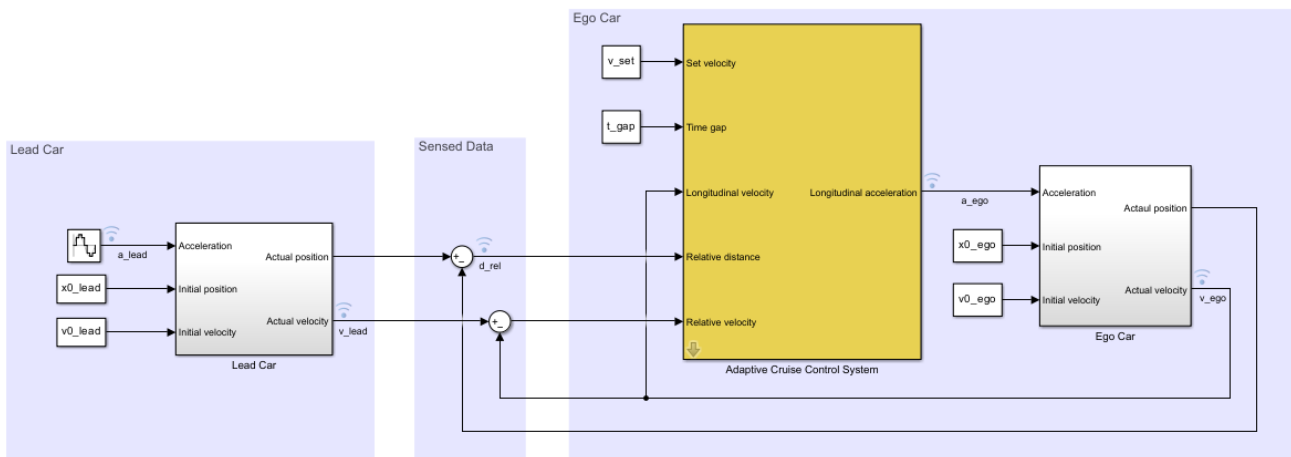
Now we are going to model the disturbance. We need a signal builder for this. We can also use a unit step but signal builder gives us more flexibility to model the disturbance. In this we model the disturbance till 300 sec as our simulation time is also 300 sec. We are modelling an increase in velocity disturbance.



Here we can see that the slope on which the vehicle is moving is increasing and after 250 seconds it remains constant. The magnitude of the velocity after 250 sec is 30. Here we expect the PID to manipulate the signal in such a way that the final speed at which the vehicle is running is approximately the set speed.

Here we auto tune the PID controller. After tuning the parameters obtained are P=1.33, I=0.26, D=-0.48.

Adaptive cruise control:



To approximate a realistic driving environment, the acceleration of the lead car varies according to a sine wave during the simulation. The Adaptive Cruise Control System block outputs an acceleration control signal for the ego car.

Defined the sample time, T_s and simulation duration, T in seconds.

- $T_s = 0.1$;
- $T = 80$;

For both the ego vehicle and the lead vehicle, the dynamics between acceleration and velocity are modelled as:

$$G = \frac{1}{s(0.5s + 1)}$$

which approximates the dynamics of the throttle body and vehicle inertia.

Specify the linear model for ego car.

- $G_{ego} = tf(1, [0.5, 1, 0]);$

Specify the initial position and velocity for the two vehicles.

- $x0_lead = 50;$ → initial position for lead car (m)
- $v0_lead = 25;$ → initial velocity for lead car (m/s)
- $x0_ego = 10;$ → initial position for ego car (m)
- $v0_ego = 20;$ → initial velocity for ego car (m/s)

Configuration of Adaptive Cruise Control System Block

The ACC system is modelled using the Adaptive Cruise Control System Block in Simulink [2],[5]. The inputs to the ACC system block are:

- Driver-set velocity V_{set}
- Time gap T_{gap}
- Velocity of the ego car V_{ego}
- Relative distance to the lead car D_{rel} (from radar)
- Relative velocity to the lead car V_{rel} (from radar)

The output for the ACC system is the acceleration of the ego car.

The safe distance between the lead car and the ego car is a function of the ego car velocity, V_{ego} :

$$D_{safe} = D_{default} + T_{gap} \times V_{ego}$$

where $D_{default}$ is the standstill default spacing and T_{gap} is the time gap between the vehicles. Specify values for $D_{default}$, in meters, and T_{gap} , in seconds.

- $t_gap = 1.4;$
- $D_default = 10;$

Specify the driver-set velocity in m/s.

- $v_{\text{set}} = 30$;

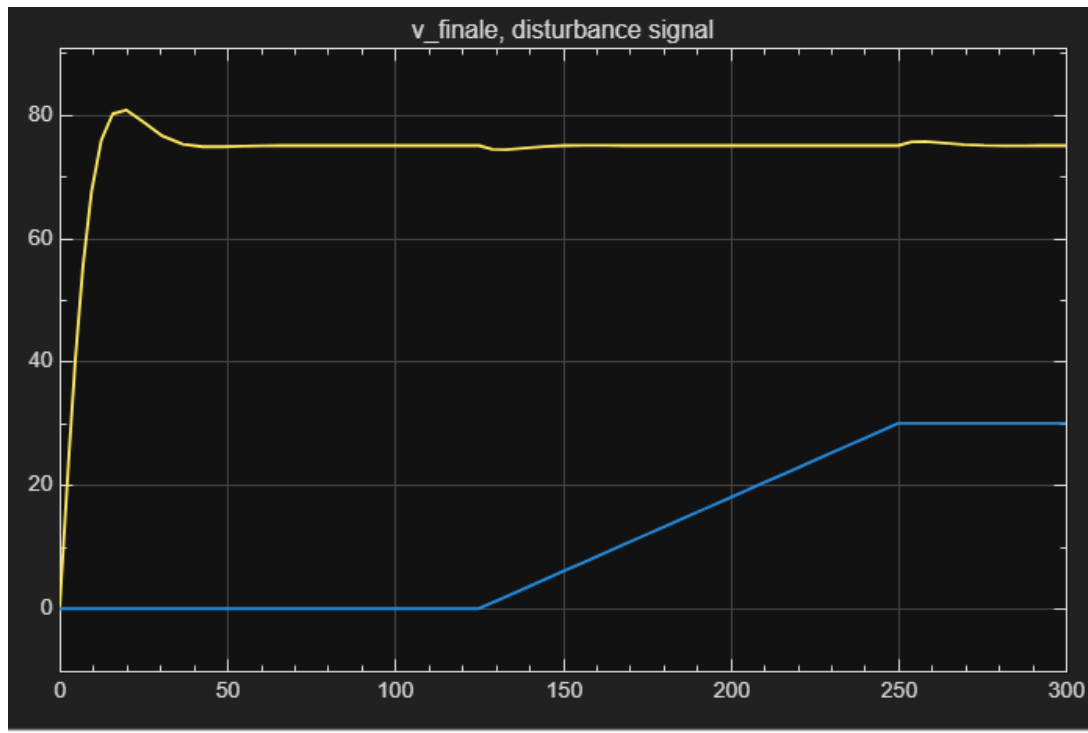
Considering the physical limitations of the vehicle dynamics, the acceleration is constrained to the range $[-3, 2]$ (m/s^2).

- $a_{\text{min_ego}} = -3$;
- $a_{\text{max_ego}} = 2$;

For this example, the default parameters of the Adaptive Cruise Control System block match the simulation parameters. If our simulation parameters differ from the default values, then we should update the block parameters accordingly.

Results and inferences:

Cruise control:



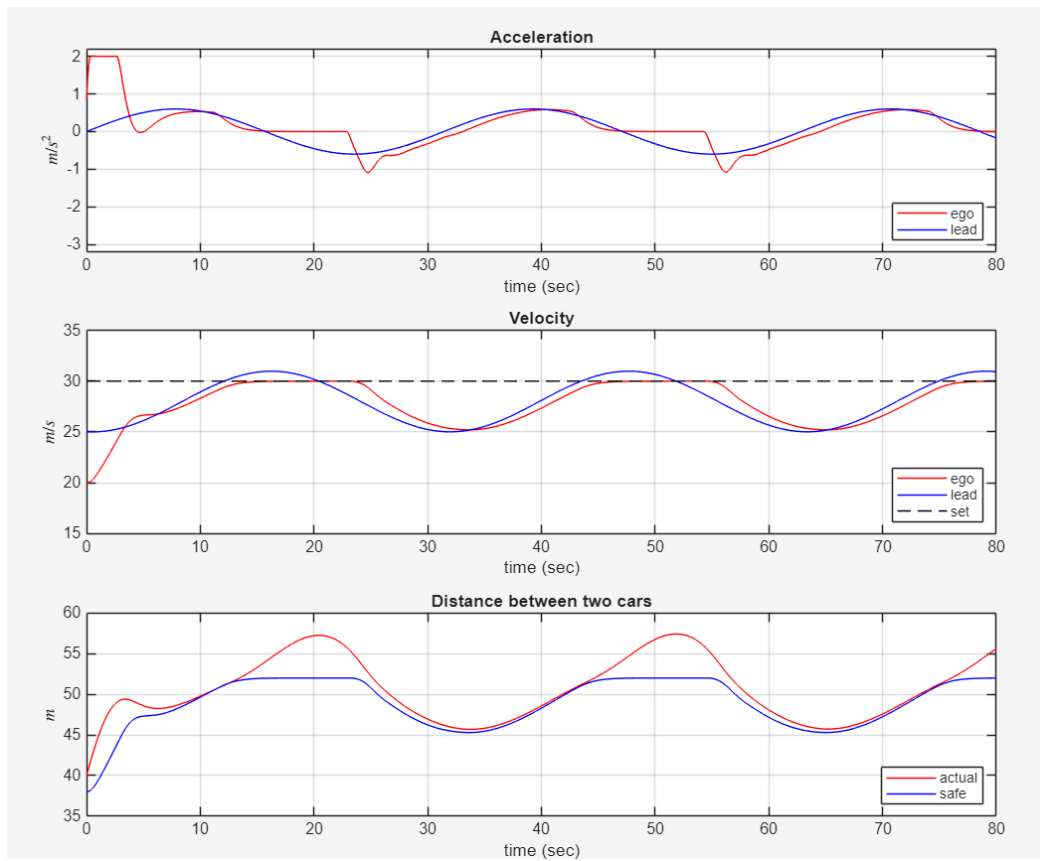
This is the curve we obtained here the initial speed was set to zero. If we want we can change the value of the initial speed in the integrator. We can see that the controller tried to reach the set value of 75 as soon as possible. In less than 25 sec it tried to reach the value of 75 and

therefore the input provided to the throttle would have been very high that is why there is a slight overshoot here the speed goes above 75 this is not desired in an ideal case.

This could come as a safety issue because in any case the speed should not go beyond what the driver set but in the simulation this is quite feasible and here we can see that at 125 sec as the disturbance signal begins to rise there is a slight dip in the speed because we took the case of simulation of an upward slope so this increase in V causes a slight dip then the PID controller takes charge and the speed is again restored to 75 kmph.

Again when disturbance signal becomes constant at 30 kmph after 250 sec previously the disturbance was continuously increasing. So that PID controller would have been increasing the value of the input to the throttle but then when it suddenly becomes constant there is a slight increase then again it comes under control after few seconds.

Adaptive cruise control:



- In the first 3 seconds, to reach the driver-set velocity, the ego car accelerates at full throttle.

- From 3 to 13 seconds, the lead car accelerates slowly. As a result, to maintain a safe distance to the lead car, the ego car accelerates with a slower rate.
- From 13 to 25 seconds, the ego car maintains the driver-set velocity, as shown in the **Velocity** plot. However, as the lead car reduces speed, the spacing error starts approaching 0 after 20 seconds.
- From 25 to 45 seconds, the lead car slows down and then accelerates again. The ego car maintains a safe distance from the lead car by adjusting its speed, as shown in the **Distance** plots [5].
- From 45 to 56 seconds, the spacing error is above 0. Therefore, the ego car achieves the driver-set velocity again.
- From 56 to 76 seconds, the deceleration/acceleration sequence from the 25 to 45 second interval is repeated.

Throughout the simulation, the controller ensures that the actual distance between the two vehicles is greater than the set safe distance [2],[5]. When the actual distance is sufficiently large, then the controller ensures that the ego vehicle follows the driver-set velocity.

Previous studies have shown that ACC significantly improves vehicle safety by maintaining safe headway and responding effectively to lead vehicle maneuvers [5].

Conclusion:

The cruise control system was successfully modelled and simulated using MATLAB/Simulink. By implementing a PID controller, the system demonstrated its ability to effectively maintain the vehicle's speed at a desired setpoint. The simulation showed minimal overshoot, a fast settling time and negligible steady-state error. The closed-loop feedback design ensured both stability and robust tracking performance. This project successfully demonstrates how classical control theory can be applied to real-world automotive systems to provide a reliable driver-assistance feature. This foundational work was then successfully extended to an Adaptive Cruise Control (ACC) system using a Model Predictive Controller (MPC), showcasing how advanced control strategies can manage more complex, real-world driving scenarios [3],[4].

Pros and Cons of the Chosen Methods:

PID Controller (for Cruise Control):

- **Pros:**
 - Simple to design and implement in Simulink.

- The method is well-understood and widely used in automotive applications.
- It provides accurate speed regulation and good rejection of disturbances.
- Tuning the PID parameters offers flexibility to achieve desired response characteristics.

- **Cons:**

- The system's performance is heavily dependent on the tuning of PID parameters.
- It does not account for rapidly changing traffic conditions.
- It has limited adaptability to nonlinearities in real vehicle dynamics.
- There is no built-in consideration for safety measures like collision avoidance.

Model Predictive Controller (MPC) (for Adaptive Cruise Control):

- **Pros:**

- Excellent at handling system constraints, such as maintaining a safe minimum distance.
- Its predictive nature is ideal for planning optimal movements in dynamic traffic.
- It can optimize for multiple objectives simultaneously (e.g., comfort, safety, and speed).

- **Cons:**

- It is more computationally complex and demanding than a PID controller.
- Performance is highly dependent on an accurate model of the vehicle's dynamics.

Possible Future Directions:

- Incorporate **nonlinear vehicle dynamics** to create a more realistic simulation, accounting for factors like engine response and actuator limits.
- Implement **fuzzy logic** for improved adaptability and robustness.
- Integrate the system with other **advanced driver assistance systems (ADAS)** like lane-keeping and automatic braking.
- Test the system with **Hardware-in-the-Loop (HIL)** or on real-time embedded systems for deployment in actual vehicles.

References

- [1] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed. New York: Springer, 2012.
- [2] A. Vahidi and A. Eskandarian, “Research advances in intelligent collision avoidance and adaptive cruise control,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 4, no. 3, pp. 143–153, Sept. 2003.
- [3] M. Buehler, K. Iagnemma, and S. Singh, Eds., *The DARPA Urban Challenge: Autonomous Vehicles in City Traffic*. Berlin, Germany: Springer, 2009.
- [4] P. Falcone, F. Borrelli, H. E. Tseng, J. Asgari, and D. Hrovat, “Predictive active steering control for autonomous vehicle systems,” *IEEE Transactions on Control Systems Technology*, vol. 15, no. 3, pp. 566–580, May 2007.
- [5] K. Yi and J. Hyun, “A study on adaptive cruise control system performance for vehicle safety enhancement,” *International Journal of Automotive Technology*, vol. 9, no. 2, pp. 231–240, Apr. 2008.